



Instituto Tecnológico de Costa Rica

Carrera:

Ingeniería en computadores

Curso:

Análisis Numérico para la Ingeniería

Portafolio Bloque 2:

Gradiente conjugado

Integrantes grupo 10:

Aldo Alessandro Bagliano Gómez

Génesis Conejo Montero

Joel David Barboza Picado

Fecha de entrega:

17 de setiembre del 2026

Contenido

1. Explicación del Método del Gradiente Conjugado	3
2. Condiciones de Aplicabilidad de la Matriz de Coeficientes	3
3. Pseudocódigo del Algoritmo	4
4. Ejemplo de Aplicación (Matriz 4×4)	6
4.1. Inicialización ($k = 0$):	6
4.2. Iteración 1:	7
4.3. Iteración 2:	8

1. Explicación del Método del Gradiente Conjugado

El método del Gradiente Conjugado (GC) es un algoritmo numérico iterativo diseñado para resolver sistemas de ecuaciones lineales de la forma $Ax = b$.

A diferencia de los métodos directos (como la Factorización QR o la Eliminación Gaussiana) que calculan la solución exacta en un número fijo de operaciones, el Gradiente Conjugado aborda la resolución mediante la optimización matemática. Transforma la ecuación $Ax = b$ en el problema de encontrar el mínimo global de la función cuadrática:

$$f(x) = \frac{1}{2} * x^T * A * x - b^T * x$$

El gradiente de esta función es $\nabla f(x) = Ax - b$, por lo que su punto crítico (donde el gradiente se anula) corresponde a la solución del sistema lineal.

En lugar de avanzar en la dirección del gradiente negativo tradicional (que puede generar pasos redundantes), el algoritmo utiliza direcciones conjugadas (ortogonales con respecto a la matriz A). Esto garantiza que la búsqueda se realice sobre direcciones independientes, evitando deshacer las correcciones realizadas en iteraciones anteriores y logrando la solución exacta en un máximo de n pasos (en aritmética exacta) para un sistema de tamaño $n \times n$.

2. Condiciones de Aplicabilidad de la Matriz de Coeficientes

Para garantizar la convergencia del algoritmo y asegurar que el problema de minimización esté bien definido, la matriz de coeficientes $A \in \mathbb{R}^{n \times n}$ debe cumplir estrictamente con dos condiciones:

- Simetría: La matriz debe ser idéntica a su transpuesta ($A = A^T$).
- Definida Positiva: Para cualquier vector no nulo $x \in \mathbb{R}^n$ ($x \neq 0$), se debe cumplir que $x^T * A * x > 0$.

Algo importante es que, si la matriz A no es simétrica ni definida positiva, la función $f(x)$ puede no tener un único mínimo global, provocando que el algoritmo falle o meza en divergencia. Para sistemas generales donde no se satisfacen estas condiciones, se suelen emplear alternativas como la Factorización QR, la cual resuelve $Ax = b$ mediante ortonormalización sin requerir simetría ni definición positiva.

3. Pseudocódigo del Algoritmo

Entradas:

A : Matriz de coeficientes ($n \times n$, simétrica y definida positiva)

b : Vector de términos independientes ($n \times 1$)

x_0 : Vector de aproximación inicial ($n \times 1$)

$iterMax$: Número máximo de iteraciones

tol : Tolerancia de convergencia

Salidas:

x_k : Aproximación de la solución ($n \times 1$)

erk : Error medido con norma 2 $\|Ax_k - b\|_2$

k : Número de iteraciones ejecutadas

$conv$: Indicador de convergencia (1 si convergió, 0 en caso contrario)

$x_k = x_0$

$r_k = b - A * x_k$

$p_k = r_k$

$k = 0$

$erk = norm(r_k, 2)$

$conv = 0$

SI $erk < tol$ ENTONCES

$conv = 1$

RETORNAR $x_k, erk, k, conv$

FIN SI

MIENTRAS $k < iterMax$ *HACER*:

$$Apk = A * pk$$

$$\alpha = \frac{rk^T * rk}{pk^T * Apk}$$

$$xk = xk + \alpha * pk$$

$$rk_nuevo = rk - \alpha * Apk$$

$$k = k + 1$$

$$erk = norm(rk_nuevo, 2)$$

SI $erk < tol$ *ENTONCES*

$$conv = 1$$

$$rk = rk_nuevo$$

RETORNAR $xk, erk, k, conv$

FIN SI

$$\beta = \frac{(rk_{nuevo}^T * rk_{nuevo})}{rk^T * rk}$$

$$pk = rk_nuevo + \beta * pk$$

$$rk = rk_nuevo$$

FIN MIENTRAS

RETORNAR $xk, erk, k, conv$

4. Ejemplo de Aplicación (Matriz 4×4)

Se desea resolver el sistema $Ax = b$ definido por:

$$A = \begin{pmatrix} 2 & -1 & 0 & 0 \\ -1 & 2 & -1 & 0 \\ 0 & -1 & 2 & -1 \\ 0 & 0 & -1 & 2 \end{pmatrix}, \quad b = \begin{pmatrix} 1 \\ 0 \\ 0 \\ 1 \end{pmatrix}$$

Tomando como estimación inicial $x_0 = \begin{pmatrix} 0 \\ 0 \\ 0 \\ 0 \end{pmatrix}$.

4.1. Inicialización ($k = 0$):

$$x_k = \begin{pmatrix} 0 \\ 0 \\ 0 \\ 0 \end{pmatrix}$$

$$r_k = b - A * x_k = \begin{pmatrix} 1 \\ 0 \\ 0 \\ 1 \end{pmatrix}$$

$$p_k = r_k = \begin{pmatrix} 1 \\ 0 \\ 0 \\ 1 \end{pmatrix}$$

$$erk = \text{norm}(r_k, 2) = \sqrt{2} \approx 1.4142$$

4.2. Iteración 1:

$$Apk = A * pk = \begin{pmatrix} 2 \\ -1 \\ -1 \\ 2 \end{pmatrix}$$

$$\alpha = \frac{rk^T * rk}{pk^T * Apk} = \frac{2}{4} = 0.5$$

$$xk = xk + \alpha * pk = \begin{pmatrix} 0.5 \\ 0 \\ 0 \\ 0.5 \end{pmatrix}$$

$$rk_nuevo = rk - \alpha * Apk = \begin{pmatrix} 0 \\ 0.5 \\ 0.5 \\ 0 \end{pmatrix}$$

$$k = k + 1 = 1$$

$$erk = \text{norm}(rk_nuevo, 2) = \sqrt{0.5^2 + 0.5^2} = 0.7071$$

$$\beta = \frac{rk_nuevo^T * rk_nuevo}{rk^T * rk} = \frac{0.5}{2} = 0.25$$

$$pk = rk_nuevo + \beta * pk = \begin{pmatrix} 0.25 \\ 0.5 \\ 0.5 \\ 0.25 \end{pmatrix}$$

$$rk = rk_nuevo = \begin{pmatrix} 0 \\ 0.5 \\ 0.5 \\ 0 \end{pmatrix}$$

4.3. Iteración 2:

$$Apk = A * pk = \begin{pmatrix} 0 \\ 0.25 \\ 0.25 \\ 0 \end{pmatrix}$$

$$\alpha = \frac{rk^T * rk}{pk^T * Apk} = \frac{0.5}{0.25} = 2$$

$$xk = xk + \alpha * pk = \begin{pmatrix} 1 \\ 1 \\ 1 \\ 1 \end{pmatrix}$$

$$rk_{nuevo} = rk - \alpha * Apk = \begin{pmatrix} 0 \\ 0 \\ 0 \\ 0 \end{pmatrix}$$

$$k = k + 1 = 2$$

$$erk = \text{norm}(rk_{nuevo}, 2) = 0$$

Como $erk < tol$, $conv = 1$, $rk = rk_{nuevo}$ y se termina el ciclo.

El método alcanza la convergencia en $k = 2$ iteraciones con solución exacta

$$xk = \begin{pmatrix} 1 \\ 1 \\ 1 \\ 1 \end{pmatrix}^T, \quad erk = 0, \quad conv = 1$$